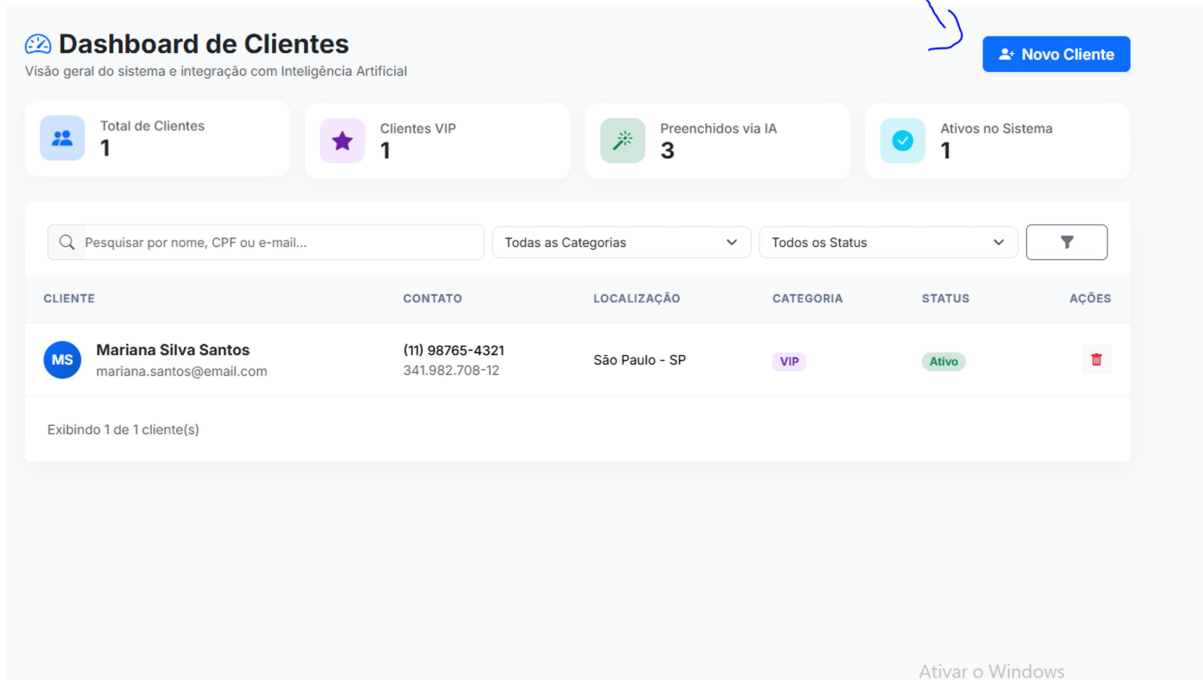


GUIA PRÁTICO

Dashboard de Clientes com Inteligência Artificial

Como construir uma interface moderna em HTML, Bootstrap e JavaScript, com preenchimento assistido por IA e arquitetura segura



Resultado final do dashboard de gestão de clientes.

Autor: Jean Paiva da Silva | Agosto de 2026

1. Visão geral do projeto

Este tutorial apresenta a construção de um protótipo de gestão de clientes que combina uma interface web moderna com preenchimento automático de formulários por Inteligência Artificial. O objetivo é demonstrar, de forma simples e didática, como a IA pode reduzir tarefas repetitivas em sistemas corporativos.

Resultado: O usuário descreve o perfil desejado, a aplicação solicita ao backend um cadastro estruturado, preenche os campos automaticamente e permite revisar os dados antes de salvar.

O que o exemplo entrega

- Dashboard responsivo com quatro indicadores (KPIs).
- Pesquisa por nome, CPF ou e-mail e filtros por categoria e status.
- Cadastro em modal com máscaras para CPF, telefone e CEP.
- Preenchimento assistido por IA a partir de uma descrição em linguagem natural.
- Tabela dinâmica, exclusão de clientes e atualização automática dos indicadores.
- Separação segura entre frontend e backend para proteger a chave da API.

Tecnologias utilizadas

Tecnologia	Responsabilidade
HTML5	Estrutura semântica da página, formulário, tabela e modal.
CSS3	Personalização visual, cartões, cores, sombras e responsividade.
Bootstrap 5.3	Grid responsivo, botões, formulários, modal e utilitários.
Bootstrap Icons	Ícones da interface.
JavaScript	Estado em memória, filtros, máscaras, renderização e comunicação HTTP.
Backend seguro	Intermediário entre o navegador e a API da OpenAI.

2. Arquitetura segura

Uma chave de API nunca deve ser colocada no HTML, no JavaScript do navegador ou em um repositório público. Qualquer pessoa consegue inspecionar o código-fonte, copiar a credencial e gerar consumo indevido.

```
NAVEGADOR -> POST /api/clientes/gerar -> BACKEND -> OPENAI
            <-      JSON do cliente  <-
```

Regra de segurança: O frontend conhece somente o endereço do backend. A chave OPENAI_API_KEY permanece no servidor, configurada por variável de ambiente ou serviço de segredos.

A documentação oficial da OpenAI recomenda evitar chaves no código ou em repositórios públicos e utilizar variáveis de ambiente ou serviços de gerenciamento de segredos.

3. Preparando o arquivo HTML

Passo 1 - Crie o documento base. Comece com o idioma português, codificação UTF-8 e a configuração de viewport para dispositivos móveis.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard de Gestão de Clientes + IA</title>
</head>
```

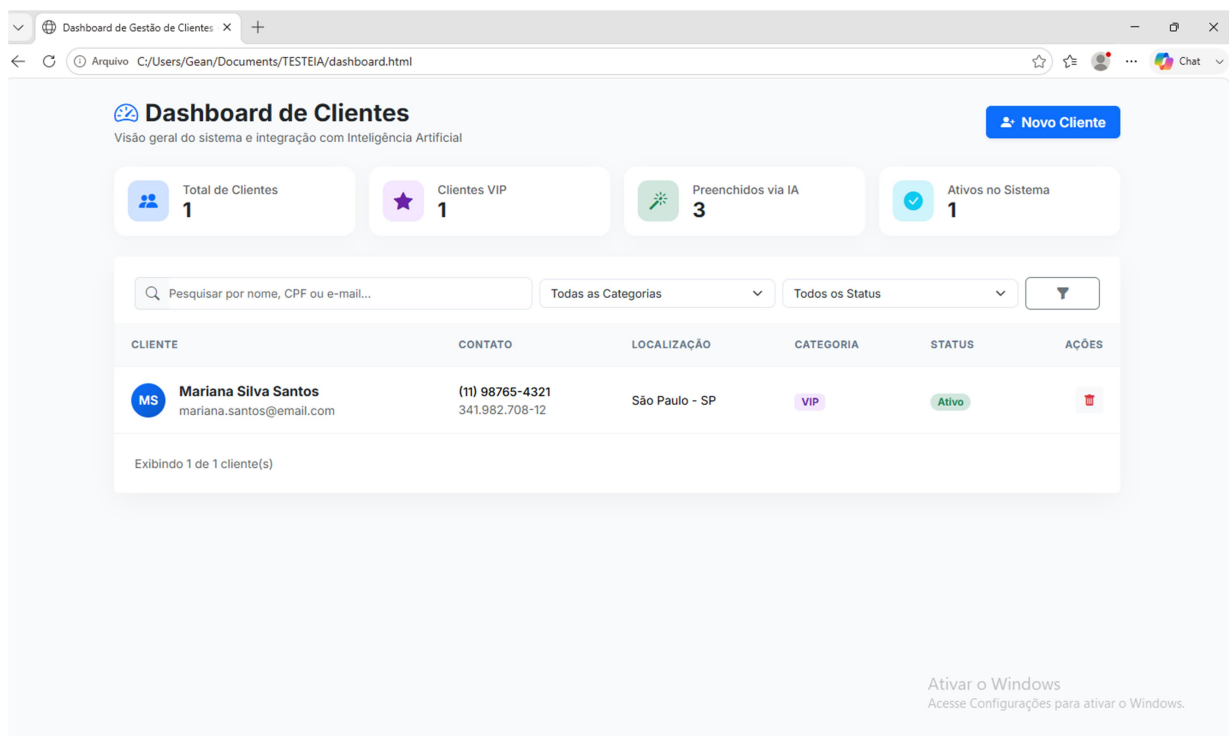
Passo 2 - Importe as dependências visuais. Bootstrap, Bootstrap Icons e a fonte Inter são carregados por CDN.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css"
rel="stylesheet">
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">
```

4. Criando o visual moderno

A identidade visual usa fundo cinza muito claro, cartões brancos, azul como cor principal, cantos arredondados e sombras discretas. O efeito de elevação nos cartões melhora a percepção de interatividade.

```
.kpi-card {
  border: none;
  border-radius: 14px;
  box-shadow: 0 4px 20px rgba(0,0,0,.03);
  transition: transform .2s ease;
}
.kpi-card:hover { transform: translateY(-3px); }
```



Cards de indicadores, filtros e tabela em uma composição responsiva.

5. Montando cabeçalho, KPIs e filtros

O cabeçalho utiliza flexbox para alinhar o título e o botão “Novo Cliente”. Em telas menores, a classe flex-column empilha os elementos; a partir do breakpoint médio, flex-md-row volta a organizá-los lado a lado.

```
<div class="d-flex flex-column flex-md-row justify-content-between align-items-md-center">
  <div>
    <h3>Dashboard de Clientes</h3>
    <p>Visão geral do sistema e integração com Inteligência Artificial</p>
  </div>
  <button class="btn btn-primary" onclick="abrirModalNovo()">Novo Cliente</button>
</div>
```

Cada KPI possui um identificador próprio. O JavaScript atualiza esses valores sempre que a lista de clientes muda:

```
kpiTotal = clientes.length
kpiVip = clientes.filter(c => c.categoria === "VIP").length
kpiIa = clientes.filter(c => c.viaIa).length
kpiAtivos = clientes.filter(c => c.status === "Ativo").length
```

6. Construindo o modal de cadastro

O modal Bootstrap XL concentra o formulário e o assistente de IA. A organização por grid permite que nome, CPF, contato e endereço ocupem larguras proporcionais sem criar uma tela pesada.

Modal de cadastro antes do preenchimento automático.

Boa experiência: A IA sugere os valores, mas o usuário mantém o controle: pode revisar, alterar ou cancelar antes de salvar. Isso é essencial em processos corporativos.

7. Modelo de dados e estado da aplicação

Para o protótipo, os clientes permanecem em um array JavaScript. Em uma solução real, essa estrutura seria substituída por chamadas a uma API e persistência em banco de dados.

```
{
  id: 101, nome: "Mariana Silva Santos",
  cpf: "341.982.708-12", email: "mariana@email.com",
  telefone: "(11) 98765-4321", cidade: "São Paulo", estado: "SP",
```

```
categoria: "VIP", status: "Ativo", viaIa: false
}
```

A propriedade `viaIa` registra a origem do preenchimento e permite calcular corretamente o indicador “Preenchidos via IA”.

8. Renderização, pesquisa e filtros

Passo 1 - Leia os filtros. O código normaliza o texto digitado e captura categoria e status.

Passo 2 - Filtre o array. Nome, e-mail e CPF participam da busca livre; categoria e status são comparações exatas.

Passo 3 - Renderize a tabela. Cada cliente filtrado cria uma linha com avatar, badges e botão de exclusão.

```
const filtrados = clientes.filter(c =>
  (c.nome.toLowerCase().includes(termo) ||
   c.email.toLowerCase().includes(termo) || c.cpf.includes(termo)) &&
  (cat === "" || c.categoria === cat) &&
  (stat === "" || c.status === stat)
);
```

Proteção adicional: Antes de inserir valores dinâmicos em `innerHTML`, use `escapeHtml()`. Isso reduz o risco de injeção de marcação ou scripts na tabela.

9. Máscaras de entrada

As funções de máscara removem caracteres não numéricos e reaplicam a formatação conforme o usuário digita. Elas melhoram a experiência, mas não substituem validação no backend.

```
function aplicarMascaraCEP(input) {
  let valor = input.value.replace(/\D/g, "");
  input.value = valor.replace(/^(\d{5})(\d)/, "$1-$2");
}
```

10. Fluxo seguro de preenchimento com IA

O navegador envia apenas a descrição do usuário ao endpoint interno. Não existe chave, token nem cabeçalho `Authorization` no HTML.

```
const API_ENDPOINT = "http://localhost:5000/api/clientes/gerar";

const response = await fetch(API_ENDPOINT, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ prompt: promptUser })
});

const clienteIA = await response.json();
```

O backend recebe o prompt, acrescenta instruções de sistema, chama a OpenAI e devolve somente o JSON do cliente. O contrato esperado contém nome, CPF, e-mail, telefone, CEP, endereço, categoria e status.

```
{
  "nome": "Cliente VIP",
  "cpf": "123.456.789-09",
  "email": "cliente.vip@example.com",
  "telefone": "+55 (11) 98765-4321",
  "cidade": "São Paulo", "estado": "SP",
  "categoria": "VIP", "status": "Ativo"
}
```

Campos preenchidos automaticamente e disponíveis para revisão humana.

11. Configurando o backend

O HTML corrigido pressupõe uma API em `http://localhost:5000/api/clientes/gerar`. No backend, configure a chave fora do código:

```
Windows PowerShell:
$env:OPENAI_API_KEY="sua-chave-nova"

Windows CMD:
set OPENAI_API_KEY=sua-chave-nova

Linux/macOS:
export OPENAI_API_KEY="sua-chave-nova"
```

Nunca faça: Não coloque uma chave real em `appsettings.json` versionado, JavaScript, README, print de tela, commit antigo ou arquivo `.env` enviado ao GitHub.

Para produção, aplique autenticação, autorização, validação de entrada, limitação de requisições, logs sem dados sensíveis, monitoramento de custos e políticas de retenção compatíveis com a empresa.

12. Salvando e excluindo clientes

No evento `submit`, `preventDefault()` evita o recarregamento da página. Um objeto é criado a partir dos campos, incluído no início do array e o dashboard é atualizado. A exclusão usa `filter()` para manter todos os clientes cujo `id` seja diferente do escolhido.

```
clientes.unshift(novoCliente);
bsModal.hide();
atualizarDashboard();

clientes = clientes.filter(c => c.id !== id);
```

13. Executando o projeto

1. **Baixe o arquivo seguro.** Utilize `dashboard-seguro.html`, nunca a versão que continha uma chave.
2. **Inicie o backend.** Confirme que a API responde em `http://localhost:5000`.
3. **Sirva o frontend.** Use a extensão Live Server do VS Code ou outro servidor HTTP local.
4. **Abra o dashboard.** Cadastre manualmente, teste filtros e exclusão.
5. **Teste a IA.** Abra “Novo Cliente”, descreva um perfil e clique em “Preencher com IA”.
6. **Revise o resultado.** Confira os dados antes de salvar; dados gerados são demonstrativos.

14. Publicando com segurança no GitHub

- Revogue a chave antiga e gere uma nova.
- Pesquise o projeto por padrões como `sk-`, `OPENAI_API_KEY` e `Authorization`.
- Inclua `.env`, `secrets.json` e arquivos locais no `.gitignore`.
- Não publique dados pessoais reais; use dados fictícios e sinalize que são demonstrativos.
- Documente que o projeto é um protótipo e que a IA requer backend.
- Adicione licença, requisitos, instruções de execução e arquitetura no README.
- Se uma chave já entrou em um commit, apenas removê-la do arquivo atual não basta: revogue-a imediatamente.

15. Limitações e próximos passos

O exemplo atual é intencionalmente simples: os dados ficam somente na memória e desaparecem ao recarregar. Uma evolução empresarial pode incluir:

- ASP.NET Core Web API como backend.
- Entity Framework Core com SQL Server, PostgreSQL ou Azure SQL.
- Autenticação com Microsoft Entra ID e perfis de acesso.
- Validação de CPF, e-mail e endereço no servidor.
- Structured Outputs ou validação de esquema para respostas da IA.
- Auditoria de alterações e aprovação humana.
- Rate limiting, telemetria, alertas de custo e observabilidade.
- Adequação à LGPD, classificação dos dados e política de retenção.

16. Conclusão

Este projeto demonstra que a Inteligência Artificial pode ser incorporada a sistemas corporativos de maneira objetiva: não para retirar o controle do usuário, mas para acelerar tarefas repetitivas e melhorar a experiência. A interface permanece responsável pela interação; o backend protege credenciais, governa a integração e valida os dados; a pessoa revisa e decide.

Mensagem principal: IA aplicada com valor exige três elementos juntos: boa experiência de uso, arquitetura segura e supervisão humana.